

Lab 4

Q1. Implement mid circle algorithm. Print the pixel coordinates of the circle centre.

```
#include <GL/glut.h>
#include <stdio.h>

int xc = 0, yc = 0, r = 100;

void display()
{
    int x = 0;
    int y = r;
    int p = 1 - r;

    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_POINTS);

    while (x <= y)
    {

        glVertex2i(xc + x, yc + y);
        glVertex2i(xc - x, yc + y);
        glVertex2i(xc + x, yc - y);
        glVertex2i(xc - x, yc - y);
        glVertex2i(xc + y, yc + x);
        glVertex2i(xc - y, yc + x);
        glVertex2i(xc + y, yc - x);
        glVertex2i(xc - y, yc - x);

        x++;

        if (p < 0)
        {
            p = p + 2 * x + 1;
        }
        else
        {
            y--;
            p = p + 2 * x - 2 * y + 1;
        }
    }

    glEnd();
    glFlush();
}

void init()
{
    glClearColor(1.0, 1.0, 1.0, 1.0);
```

```

glColor3f(0.0, 0.0, 1.0);

glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(-250, 250, -250, 250);
}

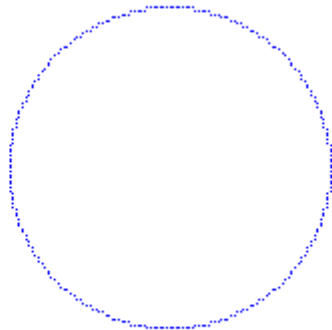
int main(int argc, char** argv)
{
    printf("Enter radius: ");
    scanf("%d", &r);

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(800, 800);
    glutCreateWindow("Midpoint Circle - No Separate Function");

    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

```



Q2. Draw circle at any given centre.

```

#include <GL/glut.h>
#include <stdio.h>

```

```

int xc, yc, r;

```

```

void display()
{
    int x = 0;

```

```

int y = r;
int p = 1 - r;

glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0, 0.0, 0.0);
glPointSize(3);

glBegin(GL_POINTS);

while (x <= y)
{

    glVertex2i(xc + x, yc + y);
    glVertex2i(xc - x, yc + y);
    glVertex2i(xc + x, yc - y);
    glVertex2i(xc - x, yc - y);
    glVertex2i(xc + y, yc + x);
    glVertex2i(xc - y, yc + x);
    glVertex2i(xc + y, yc - x);
    glVertex2i(xc - y, yc - x);

    x++;

    if (p < 0)
        p = p + 2 * x + 1;
    else
    {
        y--;
        p = p + 2 * x - 2 * y + 1;
    }
}

glEnd();
glFlush();
}

void init()
{
    glClearColor(1.0, 1.0, 1.0, 1.0);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-500, 500, -500, 500);
}

int main(int argc, char** argv)
{
    printf("Enter center (xc yc): ");
    scanf("%d %d", &xc, &yc);

    printf("Enter radius: ");
    scanf("%d", &r);
}

```

```

glutInit(&argc, argv);
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize(600, 600);
glutCreateWindow("User Input Circle - Midpoint Algorithm");

init();
glutDisplayFunc(display);
glutMainLoop();

return 0;
}

```



Q3. Draw concentric circle

```

#include <GL/glut.h>

void drawCircle(int xc, int yc, int r)
{
    int x = 0;
    int y = r;
    int p = 1 - r;

    glBegin(GL_POINTS);

    while (x <= y)
    {
        glVertex2i(xc + x, yc + y);
        glVertex2i(xc - x, yc + y);
        glVertex2i(xc + x, yc - y);
        glVertex2i(xc - x, yc - y);
        glVertex2i(xc + y, yc + x);
        glVertex2i(xc - y, yc + x);
        glVertex2i(xc + y, yc - x);
        glVertex2i(xc - y, yc - x);

        x++;
    }
}

```

```

        if (p < 0)
            p = p + 2 * x + 1;
        else
        {
            y--;
            p = p + 2 * x - 2 * y + 1;
        }
    }

    glEnd();
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    int xc = 0, yc = 0;
    glColor3f(1.0, 0.0, 0.0);
    drawCircle(xc, yc, 40);

    glColor3f(0.0, 0.0, 0.0);
    drawCircle(xc, yc, 160);

    glFlush();
}

void init()
{
    glClearColor(1.0, 1.0, 1.0, 1.0);

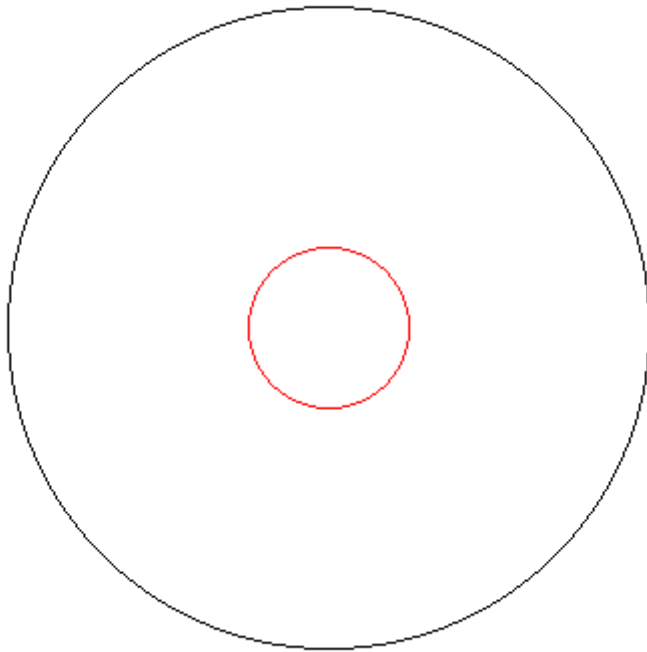
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-250, 250, -250, 250);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Concentric Circles - Midpoint Algorithm");

    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

```



Q4. Draw Multiples circles at different position

```
#include <GL/glut.h>
#include <stdio.h>

void drawCircle(int xc, int yc, int r)
{
    int x = 0;
    int y = r;
    int p = 1 - r;

    glBegin(GL_POINTS);

    while (x <= y)
    {
        glVertex2i(xc + x, yc + y);
        glVertex2i(xc - x, yc + y);
        glVertex2i(xc + x, yc - y);
        glVertex2i(xc - x, yc - y);
        glVertex2i(xc + y, yc + x);
        glVertex2i(xc - y, yc + x);
        glVertex2i(xc + y, yc - x);
        glVertex2i(xc - y, yc - x);

        x++;

        if (p < 0)
            p = p + 2 * x + 1;
        else
        {
            y--;

```

```

        p = p + 2 * x - 2 * y + 1;
    }
}

glEnd();
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0, 0.0, 0.0);
    drawCircle(0, 0, 80);

    glColor3f(0.0, 0.0, 1.0);
    drawCircle(150, 0, 50);

    glColor3f(0.0, 0.7, 0.0);
    drawCircle(-150, 0, 50);

    glFlush();
}

void init()
{
    glClearColor(1.0, 1.0, 1.0, 1.0);

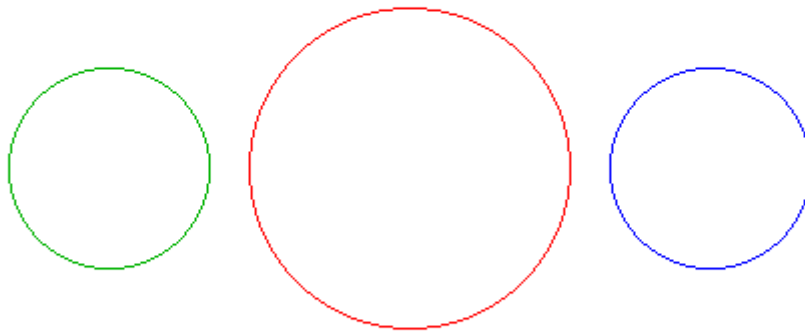
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-300, 300, -300, 300);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Multiple Circles - Midpoint Algorithm");

    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

```



Q5. Draw olympic rings

```
#include <GL/glut.h>
```

```
void drawCircle(int xc, int yc, int r)
```

```
{
```

```
    int x = 0;
```

```
    int y = r;
```

```
    int p = 1 - r;
```

```
    glBegin(GL_POINTS);
```

```
    while (x <= y)
```

```
    {
```

```
        glVertex2i(xc + x, yc + y);
```

```
        glVertex2i(xc - x, yc + y);
```

```
        glVertex2i(xc + x, yc - y);
```

```
        glVertex2i(xc - x, yc - y);
```

```
        glVertex2i(xc + y, yc + x);
```

```
        glVertex2i(xc - y, yc + x);
```

```
        glVertex2i(xc + y, yc - x);
```

```
        glVertex2i(xc - y, yc - x);
```

```
        x++;
```



```

        if (p < 0)
            p = p + 2 * x + 1;
        else
        {
            y--;
            p = p + 2 * x - 2 * y + 1;
        }
    }

    glEnd();
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(3);

    int r = 60;

    glColor3f(0.0, 0.0, 1.0);
    drawCircle(-120, 40, r);

    glColor3f(0.0, 0.0, 0.0);
    drawCircle(0, 40, r);

    glColor3f(1.0, 0.0, 0.0);
    drawCircle(120, 40, r);

    glColor3f(1.0, 1.0, 0.0);
    drawCircle(-60, -40, r);

    glColor3f(0.0, 1.0, 0.0);
    drawCircle(60, -40, r);

    glFlush();
}

void init()
{
    glClearColor(1.0, 1.0, 1.0, 1.0);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-300, 300, -200, 200);
}

int main(int argc, char** argv)

```

```
{  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);  
    glutInitWindowSize(800, 800);  
    glutCreateWindow("Olympic Rings - Midpoint Circle");  
  
    init();  
    glutDisplayFunc(display);  
    glutMainLoop();  
  
    return 0;  
}
```

